

SRI SAI RAM ENGINEERING COLLEGE
SAI LEO NAGAR, WEST TAMBARAM, CHENNAI-44
(An Autonomous Institution)



NAME

:

REGISTER NUMBER:

**24AMPW401- MACHINE LEARNING TECHNIQUES WITH
LABORATORY**

(II YEAR / IV SEM)

(BATCH: 2024–2028)

B.E CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

ACADEMIC YEAR:2025–2026



Bonafide Certificate

Register No: _____

Certified that this is the Bonafide Record of the work done by Mr./Ms

_____ in the Semester – IV of **B.E** Degree course

CSE(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING) in the

24AMPW401- MACHINE LEARNING TECHNIQUES WITH LABORATORY
during the academic year **2025-2026**.

Station:Chennai-600044

Date:

STAFF IN-CHARGE

HEAD OF THE DEPARTMENT

Submitted for University Practical Examination held on _____

at **Sri Sai Ram Engineering College**, Chennai – 600 044.

INTERNAL EXAMINER

EXTERNAL EXAMINAR

INDEX

S. No	Date	Title of the Experiment	Page No.	Signature
1		DATA PREPROCESSING TECHNIQUES		
2		SIMPLE LINEAR AND MULTIPLE LINEAR REGRESSION		
3		POLYNOMIAL REGRESSION		
4		NAÏVE BAYES AND K- NEAREST NEIGHBORS ALGORITHM		
5		DECISION TREE CLASSIFIER TECHNIQUE		
6		SUPPORT VECTOR MACHINE ALGORITHM		
7		RANDOM FOREST ALGORITHM		
8		AdaBOOST TECHNIQUE		
9		XGBOOST AND LIGHTGBM TECHNIQUE		
10		K- MEANS AND HIERARCHIAL CLUSTERING ALGORITHMS		

EX. NO.:01

DATA PREPROCESSING TECHNIQUES

DATE:

AIM: Explore Numpy and pandas libraries for data pre-processing techniques such as handling missing data and outliers, encoding to prepare data for machine learning models.

ALGORITHM:

1. The dataset is first loaded into a pandas dataframe so that it can be processed easily.
2. The numerical and categorical columns are identified based on their data types.
3. The missing values in numerical columns are replaced with the median of each columns and the missing values in categorical columns are replaced with the mode.
4. The numerical columns are then checked for outliers using the IQR method, and values outside the acceptable range the capped to the nearest boundary.
5. Finally, the categorical columns are converted into numerical form using one-hot encoding and the cleaned dataset is made ready for machine learning.

PROGRAM:

```
import numpy as np
import pandas as pd
dataset=pd.read_csv(""Data_preprocessing.csv"")
dataset.isnull().sum()

#dependent and independent variable
x=dataset.iloc[:,0:3].values
y=dataset.iloc[:,3:4].values

#import imputer and fill the null values
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
imputer= imputer.fit(x[:,1:3])
x[:,1:3]=imputer.transform(x[:,1:3])

#import label encoder and one hot encoder for categorial data
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
```

Example feature matrix (replace with your actual x)

x must exist before this

x = dataset.iloc[:, :-1].values

```
ct = ColumnTransformer(  
transformers=[('onehot', OneHotEncoder(), [0])],  
remainder='passthrough',  
)
```

```
x = ct.fit_transform(x)
```

```
x=x[:,1:]
```

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(  
x, y, test_size=0.20, random_state=0)
```

OUTPUT:

	0	1	2	3
0	0.0	0.0	27.0	52000.0
1	0.0	1.0	25.0	49000.0
2	1.0	0.0	30.0	56000.0
3	0.0	1.0	29.0	54000.0
4	1.0	0.0	35.0	58000.0
5	0.0	1.0	36.0	60000.0
6	1.0	0.0	37.0	61000.0
7	0.0	0.0	41.0	67000.0
8	0.0	0.0	38.0	65000.0
9	0.0	1.0	39.0	66000.0
10	0.0	0.0	40.0	65333.3333
11	1.0	0.0	40.0	72000.0
12	0.0	1.0	40.0	65333.3333

	0	1	2	3
7	0.0	0.0	41.0	67000.0
8	0.0	0.0	38.0	65000.0
9	0.0	1.0	39.0	66000.0
10	0.0	0.0	40.0	65333.3333
11	1.0	0.0	40.0	72000.0
12	0.0	1.0	40.0	65333.3333
13	1.0	0.0	44.0	72000.0
14	0.0	1.0	48.0	79000.0
15	1.0	0.0	47.0	78000.0
16	0.0	0.0	50.0	83000.0
17	0.0	1.0	52.0	85000.0
18	1.0	0.0	38.77777777	61000.0
19	0.0	0.0	38.77777777	58000.0

	0
0	No
1	Yes
2	No
3	No

	0
0	Yes
1	No
2	Yes
3	No
4	Yes
5	No
6	Yes
7	Yes
8	No
9	Yes
10	No
11	No
12	No

x_test - NumPy object array

	0	1	2	3
0	1.0	0.0	38.777777...	61000.0
1	0.0	1.0	25.0	49000.0
2	0.0	0.0	38.777777...	58000.0
3	0.0	0.0	38.0	65000.0

x_train - NumPy object array

	0	1	2	3
0	0.0	0.0	40.0	65333.3333...
1	0.0	1.0	52.0	85000.0
2	1.0	0.0	37.0	61000.0
3	1.0	0.0	44.0	72000.0
4	1.0	0.0	35.0	58000.0
5	1.0	0.0	30.0	56000.0
6	0.0	1.0	36.0	60000.0
7	0.0	1.0	48.0	79000.0
8	0.0	1.0	39.0	66000.0
9	0.0	0.0	41.0	67000.0
10	0.0	0.0	50.0	83000.0
11	1.0	0.0	40.0	72000.0
12	0.0	1.0	29.0	54000.0

y - NumPy object array

	0
0	Yes
1	Yes
2	No
3	No
4	Yes
5	Yes
6	Yes
7	Yes
8	No
9	No
10	Yes
11	No
12	Yes

y - NumPy object array

	0
7	Yes
8	No
9	No
10	Yes
11	No
12	Yes
13	No
14	Yes
15	Yes
16	No
17	No
18	No
19	No

y_train - NumPy object array

	0
3	No
4	Yes
5	No
6	Yes
7	Yes
8	No
9	Yes
10	No
11	No
12	No
13	Yes
14	Yes
15	Yes

RESULT: Thus the program to explore Numpy and pandas libraries for data pre-processing techniques such as handling missing data and outliers, encoding to prepare data for machine learning models has been successfully executed and the output has been verified.

EX. NO.:02

SIMPLE LINEAR AND MULTIPLE LINEAR REGRESSION

DATE:

AIM: To implement linear regression to predict a continuous target variable (Happiness score) and explore how multiple linear regression can capture the relationship between multiple predictors and the target using the GHI dataset.

ALGORITHM:

1. Import the required libraries such as numpy, pandas, matplotlib and scikit-learn.
2. Load the dataset (GHI_Report.csv) from the given file path.
3. Select H_score as the target variable
4. Select Economy for simple regression and economy, fam, Health, Freedom for multiple regression.
5. Train the simple linear regression model.
6. Predict values and calculate RMSE and R-square score.
7. Train the multiple linear regression model.
8. Plot the simple linear regression graph.
9. Plot the multiple linear regression graph.
10. Display the results and end the program.

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
df = pd.read_csv("C:/Users/Admin/OneDrive/Desktop/dataset/GHI_Report.csv")
target_column = 'H_Score'
simple_feature = ['Economy']
multiple_features = ['Economy', 'Fam', 'Health', 'Freedom']
Y = df[target_column].values
# SIMPLE LINEAR REGRESSION
X_simple = df[simple_feature].values
```

```

simple_model = LinearRegression()
simple_model.fit(X_simple, Y)
Y_simple_pred = simple_model.predict(X_simple)
rmse_simple = np.sqrt(mean_squared_error(Y, Y_simple_pred))
r2_simple = simple_model.score(X_simple, Y)
print("\n--- Simple Linear Regression ---")
print("Coefficient (Slope):", simple_model.coef_[0])
print("Intercept:", simple_model.intercept_)
print("RMSE:", rmse_simple)
print("R2 Score:", r2_simple)
# Plot Simple Regression
plt.figure()
plt.scatter(X_simple, Y, marker='*', label="Actual Data")
plt.plot(X_simple, Y_simple_pred, 'b-', label="Regression Line")
plt.xlabel("Economy")
plt.ylabel("H_Score")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()

```

MULTIPLE LINEAR REGRESSION

```

X_multi = df[multiple_features].values
multi_model = LinearRegression()
multi_model.fit(X_multi, Y)
Y_multi_pred = multi_model.predict(X_multi)
rmse_multi = np.sqrt(mean_squared_error(Y, Y_multi_pred))
r2_multi = multi_model.score(X_multi, Y)
print("\n--- Multiple Linear Regression ---")
for feature, coef in zip(multiple_features, multi_model.coef_):
    print(f"Coefficient for {feature}: {coef}")
print("Intercept:", multi_model.intercept_)
print("RMSE:", rmse_multi)
print("R2 Score:", r2_multi)
# Plot Multiple Regression (Actual vs Predicted)

```



```

plt.figure()
plt.scatter(Y, Y_multi_pred, marker='o')
plt.xlabel("Actual H_Score")
plt.ylabel("Predicted H_Score")
plt.title("Multiple Linear Regression: Actual vs Predicted")
plt.plot([Y.min(), Y.max()], [Y.min(), Y.max()], 'r--')
plt.show()
print("\nVisualization completed successfully ✔")

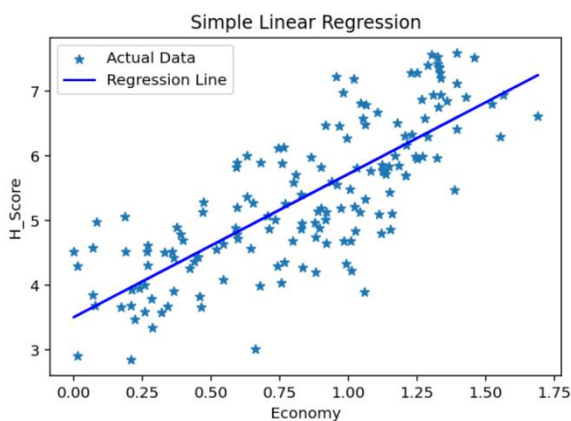
```

OUTPUT:

```

--- Simple Linear Regression ---
Coefficient (Slope): 2.2182271193090632
Intercept: 3.4988096598243628
RMSE: 0.7128770054348368
R2 Score: 0.6099071541531225

```

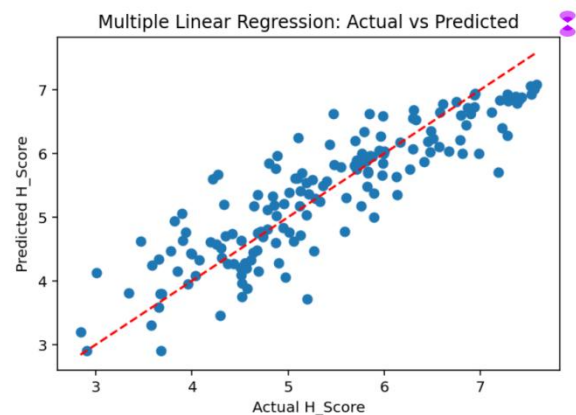


```

--- Multiple Linear Regression ---
Coefficient for Economy: 0.882587563629816
Coefficient for Fam: 1.358105547702774
Coefficient for Health: 0.9959448475728669
Coefficient for Freedom: 1.770713573126382
Intercept: 1.8963411367389158
RMSE: 0.5474062000828458
R2 Score: 0.7699839204155501

Visualization completed successfully ✔

```



RESULT: Thus the program to To implement linear regression to predict a continuous target variable (Happiness score) and explore how multiple linear regression can capture the relationship between multiple predictors and the target using the GHI dataset. The study includes model training, validation and performance evaluation has been successfully executed and the output is verified.

EX. NO.:03

POLYNOMIAL REGRESSION

DATE:

AIM: To apply Polynomial Regression to model non-linear relationships and investigate how polynomial degree impacts model fitting on GHI data, avoiding overfitting and underfitting.

ALGORITHM:

1. Import libraries and load the GHI dataset.
2. Select Economy as independent variable (X) and H_Score as dependent variable (Y).
3. Split data into 80% training set and 20% test set.
4. Define polynomial degrees to test: 1, 2, 3, and 4.
5. For each degree, convert data to polynomial features and train a Linear Regression model.
6. Calculate Train R^2 , Test R^2 , and RMSE for each model.
7. Determine model fit as overfitting, underfitting, or good fit based on performance.
8. Store results for all polynomial degrees.
9. Identify the best degree using highest Test R^2 .
10. Visualize polynomial curves using a 2×2 grid with training and test data.
11. Plot Train R^2 vs Test R^2 and RMSE comparison graphs.
12. Display the best model with optimal balance between accuracy and complexity.

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Load and split data
```

```

data = pd.read_csv('GHI_Report.csv')

X = data['Economy'].values.reshape(-1, 1)

Y = data['H_Score'].values

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# Test polynomial degrees

degrees = [1, 2, 3, 4]

results = []

print('='*60)

print('POLYNOMIAL REGRESSION ANALYSIS')

print('='*60)

for deg in degrees:

    poly = PolynomialFeatures(degree=deg)

    model = LinearRegression().fit(poly.fit_transform(X_train), Y_train)

    train_r2 = model.score(poly.transform(X_train), Y_train)

    test_r2 = model.score(poly.transform(X_test), Y_test)

    test_rmse = np.sqrt(mean_squared_error(Y_test, model.predict(poly.transform(X_test))))

    status = 'OVERFITTING' if train_r2 - test_r2 > 0.15 else ('UNDERFITTING' if test_r2 < 0.5 else
'GOOD FIT')

    results.append({'deg': deg, 'train_r2': train_r2, 'test_r2': test_r2, 'rmse': test_rmse, 'poly': poly,
'model': model})

    print(f'Degree {deg}: Train R²={train_r2:.4f}, Test R²={test_r2:.4f}, RMSE={test_rmse:.4f}
[{'status}'])

best = max(results, key=lambda x: x['test_r2'])

print(f'\nBest Degree: {best["deg"]} (Test R² = {best["test_r2"]:.4f})')

print('='*60)

# Plot polynomial fits (2x2 grid for 4 plots)

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

X_range = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)

```

```

for idx, r in enumerate(results):
    ax = axes.flatten()[idx]

    Y_pred = r['model'].predict(r['poly'].transform(X_range))

    ax.scatter(X_train, Y_train, c='blue', alpha=0.5, s=30, label='Train')
    ax.scatter(X_test, Y_test, c='red', alpha=0.7, s=50, marker='*', label='Test')

    ax.plot(X_range, Y_pred, 'g-', linewidth=2, label=f'Deg {r["deg"]}')

    ax.set_title(f'Degree {r["deg"]} (R2= {r["test_r2"]:.3f})')

    ax.legend()

    ax.grid(True, alpha=0.3)

plt.tight_layout()

plt.show()

```

Performance comparison

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

degs = [r['deg'] for r in results]

ax1.plot(degs, [r['train_r2'] for r in results], 'o-', label='Train R2', linewidth=2, markersize=8)
ax1.plot(degs, [r['test_r2'] for r in results], 's-', label='Test R2', linewidth=2, markersize=8)
ax1.axvline(x=best['deg'], color='red', linestyle='--', alpha=0.5, label='Best')

ax1.set_xlabel('Polynomial Degree')
ax1.set_ylabel('R2 Score')
ax1.set_title('R2 vs Degree')
ax1.legend()

ax1.grid(True, alpha=0.3)

ax2.plot(degs, [r['rmse'] for r in results], 'o-', color='orange', linewidth=2, markersize=8)
ax2.axvline(x=best['deg'], color='red', linestyle='--', alpha=0.5, label='Best')

ax2.set_xlabel('Polynomial Degree')
ax2.set_ylabel('RMSE')
ax2.set_title('RMSE vs Degree')

```

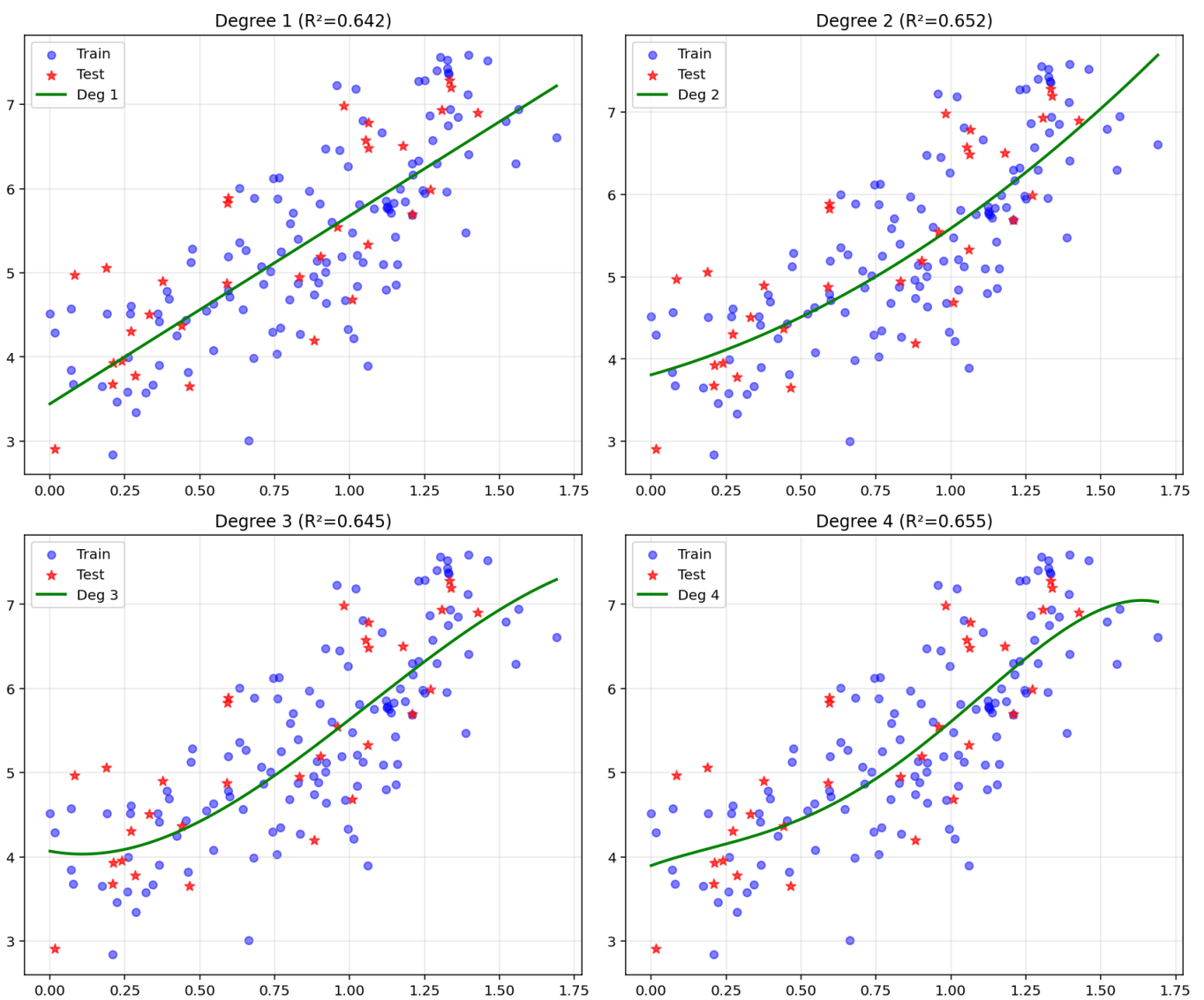
```
ax2.legend()
```

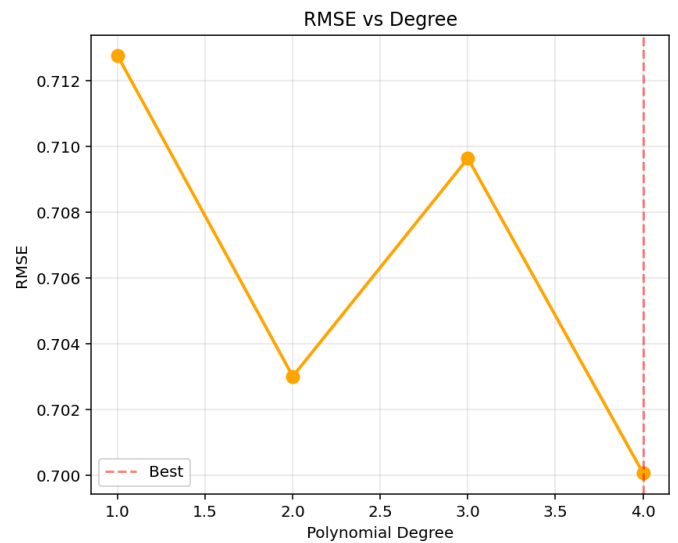
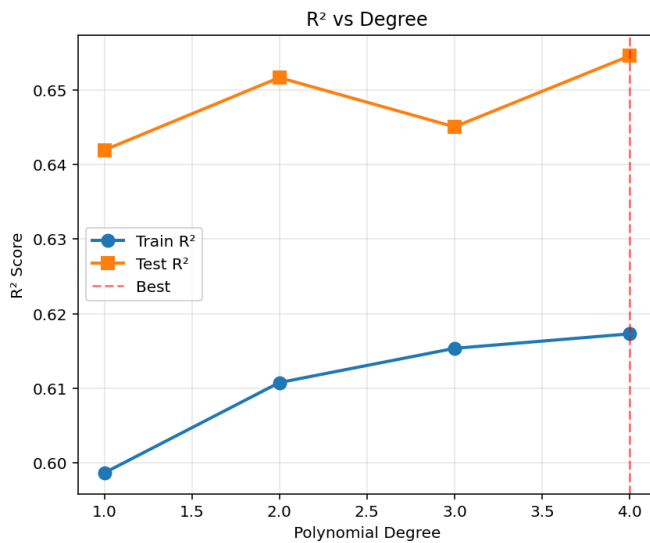
```
ax2.grid(True, alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:





```
In [4]: %runfile C:/Users/Admin/.spyder-py3/temp.py --wdir
=====
POLYNOMIAL REGRESSION ANALYSIS
=====
Degree 1: Train R²=0.5986, Test R²=0.6419, RMSE=0.7128 [GOOD FIT]
Degree 2: Train R²=0.6108, Test R²=0.6517, RMSE=0.7030 [GOOD FIT]
Degree 3: Train R²=0.6153, Test R²=0.6450, RMSE=0.7096 [GOOD FIT]
Degree 4: Train R²=0.6173, Test R²=0.6546, RMSE=0.7001 [GOOD FIT]

Best Degree: 4 (Test R² = 0.6546)
=====
```

RESULT: Thus the program to apply Polynomial Regression to model non-linear relationships and investigate how polynomial degree impacts model fitting on GHI data, avoiding overfitting and underfitting.

EX.NO.:04

NAÏVE BAYES AND K- NEAREST NEIGHBORS ALGORITHM

DATE:

AIM: To learn how to apply Naïve Bayes and k-nearest neighbors (KNN) supervised learning technique to identify level of experience using the GHI_Report Dataset.

ALGORITHM:

1. Import libraries; pandas, matplotlib, sklearn modules (train_test_split, StandardScaler, GaussianNB, KNN, metrics).
2. Read the dataset using pd.read_csv()
3. Convert H_score into categorical values: Low, Medium, High.
4. Count the number of samples in each category. Plot a bargraph to show distribution
5. Feature(X): Economy, Fam, Health, Freedom. Target(y): Experience
6. Divide data into Training set (80%) and Testing set (20%).
7. Apply StandardScaler, Fit on training data and transform both train and test sets.
8. Create Naïve Bayes model (GaussianNB) and KNN model (KNeighborsClassifier)
9. Fit both models using training data.
10. Predict Experience level using Naïve Bayes and KNN algorithms.

PROGRAM:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay

# Load dataset

data = pd.read_csv(r"C:\Users\Admin\Downloads\GHI_Report.csv")

# Create Experience Level from H_Score
```

```
def level(score):  
    if score <= 5.5:  
        return "Low"  
    elif score <= 6.5:  
        return "Medium"  
    else:  
        return "High"  
  
data["Experience"] = data["H_Score"].apply(level)
```

Plot 1: Experience distribution

```
data["Experience"].value_counts().plot(kind="bar")  
  
plt.title("Experience Level Distribution")  
  
plt.xlabel("Experience Level")  
  
plt.ylabel("Count")  
  
plt.show()
```

Features and target

```
X = data[['Economy','Fam','Health','Freedom']]  
  
y = data['Experience']
```

Split dataset

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.3)
```

Naive Bayes Model

```
nb = GaussianNB()  
  
nb.fit(X_train,y_train)  
  
nb_pred = nb.predict(X_test)
```

KNN Model

```
knn = KNeighborsClassifier(n_neighbors=3)
```



```
knn.fit(X_train,y_train)

knn_pred = knn.predict(X_test)

nb_acc = accuracy_score(y_test,nb_pred)

knn_acc = accuracy_score(y_test,knn_pred)

print("Naive Bayes Accuracy:",nb_acc)

print("KNN Accuracy:",knn_acc)
```

Accuracy Comparison Plot

```
models = ["Naive Bayes","KNN"]

accuracy = [nb_acc,knn_acc]

plt.bar(models,accuracy)

plt.title("Model Accuracy Comparison")

plt.ylabel("Accuracy")

plt.show()
```

Confusion Matrix (Naive Bayes)

```
cm_nb = confusion_matrix(y_test,nb_pred)

disp_nb = ConfusionMatrixDisplay(confusion_matrix=cm_nb)

disp_nb.plot()

plt.title("Naive Bayes Confusion Matrix")

plt.show()
```

Confusion Matrix (KNN)

```
cm_knn = confusion_matrix(y_test,knn_pred)

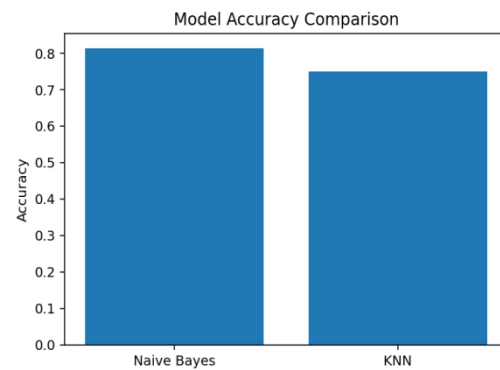
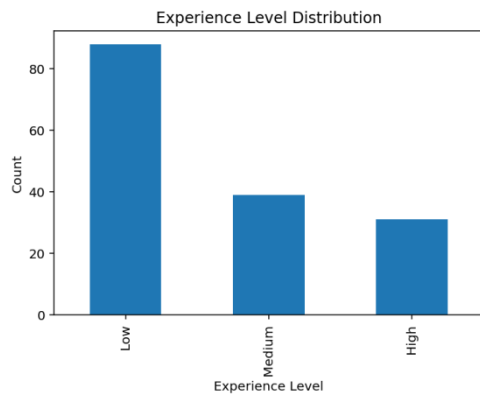
disp_knn = ConfusionMatrixDisplay(confusion_matrix=cm_knn)

disp_knn.plot()

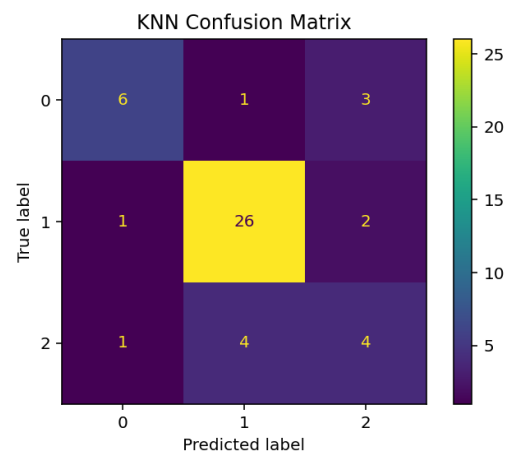
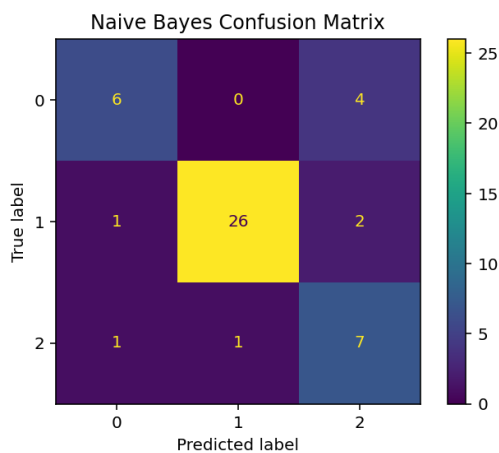
plt.title("KNN Confusion Matrix")

plt.show()
```

OUTPUT:



```
Naive Bayes Accuracy: 0.8125  
KNN Accuracy: 0.75  
Naive Bayes Prediction: ['Medium']  
KNN Prediction: ['High']
```



RESULT: Thus the program to learn how to apply Naïve Bayes and k- nearest neighbours (KNN) supervised learning technique to identify level of experience has been successfully executed and the output has been verified.

EX.NO.:05

DECISION TREE CLASSIFIER TECHNIQUE

DATE:

AIM: To implement a decision tree classifier technique to identify whether the cars require service or not and plot the confusion matrix using seaborn library using the Toyota dataset.

ALGORITHM:

1. Import required libraries for data processing, modelling and visualization. These include pandas, numpy, sklearn, matplotlib and seaborn.
2. Load the Toyota dataset into a dataframe.
3. Clean the data by replacing invalid values and converting columns to numeric.
4. Handle missing values using median for numeric data and formatted file for others.
5. Create the target variable service_required based on KM and age conditions.
6. Encode categorical data like FuelType using one-hot encoding.
7. Split the data into feature(X) and target(y), then into training and testing sets.
8. Train a Decision Tree model with limited depth. This helps controlling overfitting.
9. Predict results on test data and evaluate using confusion matrix and report.
10. Visualize results using a heatmap.

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# Load Toyota dataset
df = pd.read_csv(r"C:\Users\Admin\Downloads\Toyota.csv")

# Clean Data
```

```

df.replace("????", np.nan, inplace=True)

# Convert HP, Price, Doors to numeric
df['HP'] = pd.to_numeric(df['HP'], errors='coerce')
df['Price'] = pd.to_numeric(df['Price'], errors='coerce')
df['Doors'] = df['Doors'].replace({'three': 3, 'four': 4, 'five': 5})
df['Doors'] = pd.to_numeric(df['Doors'], errors='coerce')

# Fill numeric NaNs with median
df.fillna(df.median(numeric_only=True), inplace=True)

# Forward fill remaining missing categorical values
df.ffill(inplace=True)

# Create Service Required Target
df['service_required'] = np.where((df['KM'] > 50000) | (df['Age'] > 30), 1, 0)

# One-Hot Encode Categorical Columns
df = pd.get_dummies(df, columns=['FuelType'], drop_first=True)

# Features & Target
X = df.drop('service_required', axis=1)
y = df['service_required']

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train Decision Tree
model = DecisionTreeClassifier(max_depth=5, random_state=42)
model.fit(X_train, y_train)

# Plot Decision Tree
plt.figure(figsize=(20,10))

plot_tree(model, filled=True, feature_names=X.columns, class_names=['No Service', 'Service'],
fontsize=10)

plt.title("Decision Tree for Toyota Service Prediction")

plt.show()

# Predict

```

```

y_pred = model.predict(X_test)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:\n", cm)

print("\nClassification Report:\n")

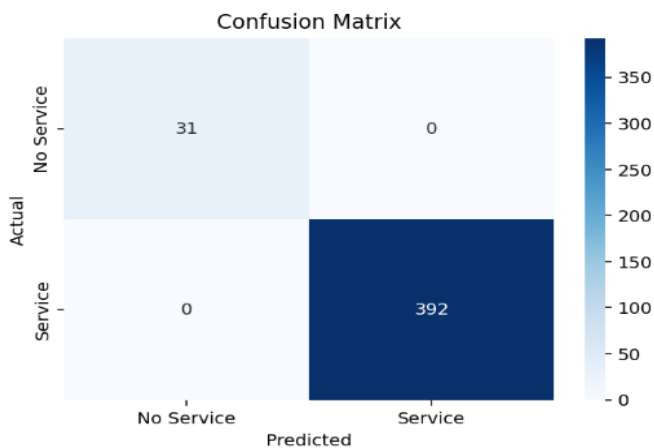
print(classification_report(y_test, y_pred))

# Plot Confusion Matrix
plt.figure(figsize=(6,4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Service', 'Service'])

```

OUTPUT:



```

Confusion Matrix:
[[ 31  0]
 [ 0 392]]

Classification Report:

              precision    recall  f1-score   support

     0       1.00      1.00      1.00         31
     1       1.00      1.00      1.00        392

 accuracy          1.00          1.00          1.00        423
 macro avg          1.00          1.00          1.00        423
 weighted avg          1.00          1.00          1.00        423

```

Decision Tree Structure



RESULT: Thus the program to implement a decision tree classifier technique to identify whether the cars require service or not and plot the confusion matrix using seaborn library has been successfully executed and the output is verified.

EX. NO.:06

SUPPORT VECTOR MACHINE ALGORITHM

DATE:

AIM: To implement a Support Vector Machine technique to identify user behaviour classes and to plot the confusion matrix using the seaborn library, using the user_behaviour dataset.

ALGORITHM:

1. Import required libraries such as pandas, numpy, sklearn, matplotlib, and seaborn for data handling, modeling, and visualization.
2. Load the dataset user_behavior_dataset.csv into a dataframe using pandas.
3. Display the first few rows and column names to understand the structure of the dataset.
4. Handle missing values by removing rows with null entries using dropna() to ensure clean data.
5. Convert categorical variables into numerical format using one-hot encoding with pd.get_dummies().
6. Separate the dataset into feature matrix (X) and target variable (y), assuming the last column is the target.
7. Convert the target variable into categorical type to avoid errors during model training.
8. Split the dataset into training and testing sets using an 80:20 ratio with train_test_split().
9. Train a Support Vector Machine (SVM) model using the RBF kernel on the training data.
10. Predict the test results and evaluate the model using classification report and confusion matrix, and visualize it using a heatmap.

PROGRAM:

Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import confusion_matrix, classification_report
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

Load Dataset

```
df = pd.read_csv("user_behavior_dataset.csv")
```

Basic Info

```
print("First 5 rows:\n", df.head())
```

```
print("\nColumns:\n", df.columns)
```

Handle Missing Values

```
df = df.dropna()
```

Convert Categorical Data

```
df = pd.get_dummies(df, drop_first=True)
```

Define Features & Target

```
target_column = df.columns[-1]
```

```
X = df.drop(target_column, axis=1)
```

```
y = df[target_column]
```

Convert target to categorical

```
y = y.astype('category')
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

Train SVM Model

```
model = SVC(kernel='rbf')
```

```
model.fit(X_train, y_train)
```


Predictions

```
y_pred = model.predict(X_test)
```

Fix Index Issue

```
y_test = y_test.reset_index(drop=True)
```

```
y_pred = pd.Series(y_pred)
```

Evaluation

```
print("\nClassification Report:\n")
```

```
print(classification_report(y_test, y_pred))
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(6, 4))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

```
plt.xlabel("Predicted")
```

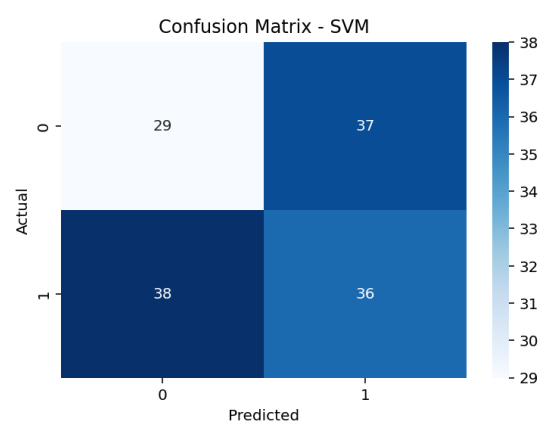
```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix - SVM")
```

```
plt.show()
```

OUTPUT:

Classification Report:				
	precision	recall	f1-score	support
0.0	0.43	0.44	0.44	66
1.0	0.49	0.49	0.49	74
accuracy			0.46	140
macro avg	0.46	0.46	0.46	140
weighted avg	0.46	0.46	0.46	140



RESULT: Thus the program to implement a Support Vector Matrix to identify user behavior classes and Plot the confusion Matrix using seaborn library has been successfully executed and the output is verified.

EX. NO.:07

RANDOM FOREST ALGORITHM

DATE:

AIM: To implement the Random Forest algorithm to identify Classification of users based on driving habits and Plot the confusion Matrix using seaborn library using the ev_charging_patterns dataset.

ALGORITHM:

1. Import required libraries such as pandas, sklearn, seaborn, and matplotlib for data processing, modeling, and visualization.
2. Load the dataset ev_charging_patterns.csv into a dataframe.
3. Select the target column (User Type) for classification.
4. Separate the dataset into feature variables (X) and target variable (y).
5. Convert categorical features into numerical format using one-hot encoding.
6. Split the dataset into training and testing sets using an 80:20 ratio.
7. Train the Random Forest classifier with a specified number of trees.
8. Predict the output using the test dataset.
9. Evaluate the model using accuracy score and confusion matrix.
10. Visualize the confusion matrix using a heatmap for better understanding.

PROGRAM:

Import Libraries

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

Load Dataset

```
df = pd.read_csv("/Users/sairam/Downloads/ev_charging_patterns.csv")
```

Select Target Column

```
target_column = "User Type"
```

Separate Features and Target

```
X = df.drop(target_column, axis=1)
```

```
y = df[target_column]
```

Convert Categorical Data

```
X = pd.get_dummies(X)
```

Split Data

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42)
```

Train Random Forest Model

```
model = RandomForestClassifier(n_estimators=100, random_state=42)  
model.fit(X_train, y_train)
```

Prediction

```
y_pred = model.predict(X_test)
```

Accuracy and Confusion Matrix

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
cm = confusion_matrix(y_test, y_pred)
```

Plot Confusion Matrix

```
plt.figure()
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=model.classes_,
```

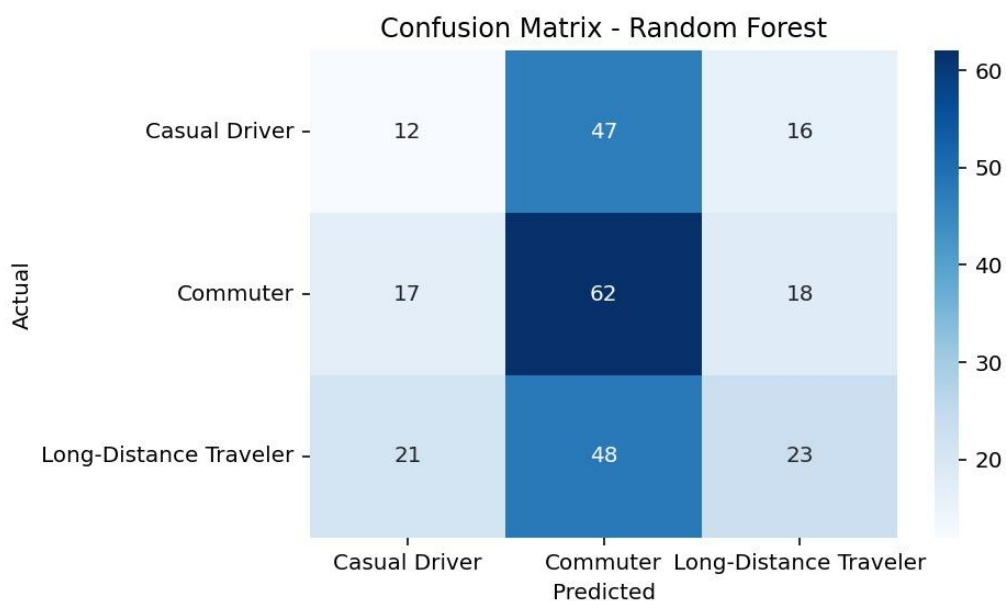
```
            yticklabels=model.classes_)
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")  
  
plt.title("Confusion Matrix - Random Forest")  
  
plt.show()
```

OUTPUT:

```
In [1]: %runfile C:/Users/sanja/Downloads/mlt7.py --wdir  
Accuracy: 0.36742424242424243
```



RESULT: Thus the program to implement Random Forest Algorithm to identify Classification of users based on driving habits and Plot the confusion Matrix using seaborn library has been successfully executed and the output is verified.

EX.NO.:08

AdaBOOST TECHNIQUE

DATE:

AIM: To implement the AdaBoost technique to Classify loan approval and loan not approval and Plot the confusion Matrix using seaborn library using credit_risk_dataset.

ALGORITHM:

1. Import required libraries such as pandas, sklearn, seaborn, and matplotlib for data processing and visualization.
2. Load the dataset credit_risk_dataset.csv into a dataframe.
3. Convert categorical variables into numerical format using one-hot encoding.
4. Separate the dataset into features (X) and target variable (loan_status).
5. Handle missing values by replacing them with the mean using an imputer.
6. Split the dataset into training and testing sets using a 70:30 ratio.
7. Train the AdaBoost classifier with a specified number of estimators.
8. Predict the results using the test dataset.
9. Evaluate the model using accuracy score and confusion matrix.
10. Visualize the confusion matrix using a heatmap.

PROGRAM:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import confusion_matrix, accuracy_score

from sklearn.impute import SimpleImputer

import seaborn as sns

import matplotlib.pyplot as plt

# Load dataset

data = pd.read_csv("credit_risk_dataset.csv")
```

Convert categorical data

```
data = pd.get_dummies(data, drop_first=True)
```

Split features & target

```
X = data.drop('loan_status', axis=1)
```

```
y = data['loan_status']
```

Handle Missing Values

```
imputer = SimpleImputer(strategy='mean')
```

```
X = imputer.fit_transform(X)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=42)
```

Apply AdaBoost

```
model = AdaBoostClassifier(n_estimators=50, random_state=42)
```

```
model.fit(X_train, y_train)
```

Predictions

```
y_pred = model.predict(X_test)
```

Accuracy and Confusion Matrix

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
cm = confusion_matrix(y_test, y_pred)
```

Visualization

```
sns.heatmap(cm, annot=True, fmt='d')
```

```
plt.xlabel("Predicted")
```

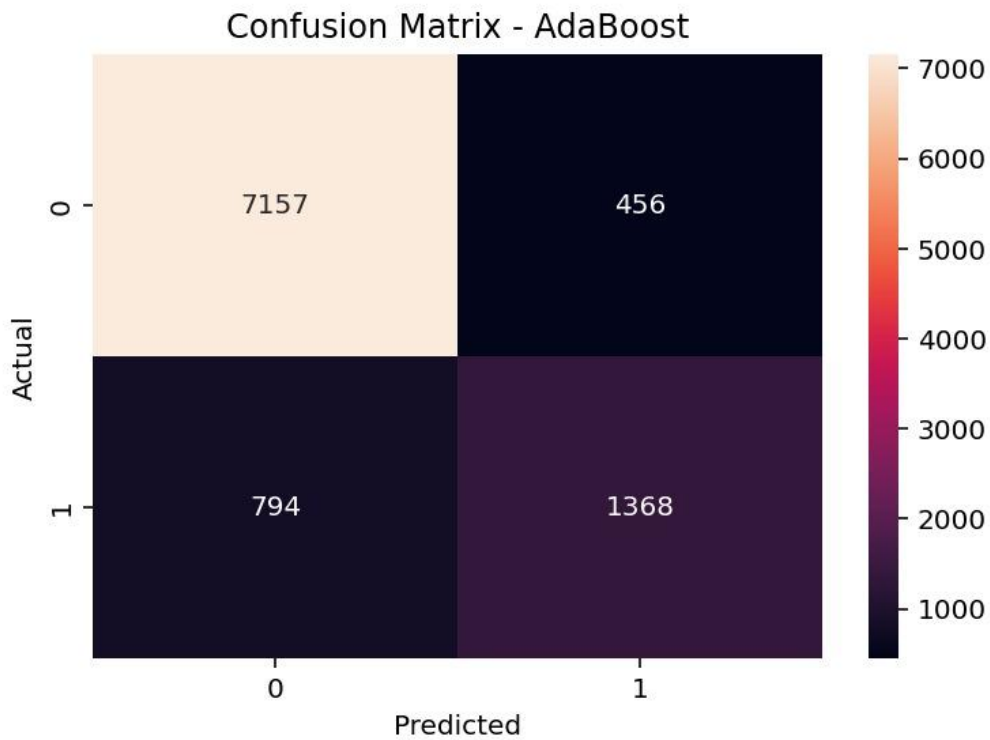
```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix - AdaBoost")
```

```
plt.show()
```

OUTPUT:

```
In [1]: %runfile C:/Users/sanja/Downloads/mlt8.py --wdir
Accuracy: 0.8721227621483376
```



RESULT: Thus the program to AdaBoost technique to Classify loan approval and loan not approval and Plot the confusion Matrix using seaborn library has been successfully executed and the output is verified.

EX. NO.:09

XGBOOST AND LIGHTGBM TECHNIQUE

DATE:

AIM: To implement the XGBoost and LightGBM technique to Classify Waitlist and admit in Wharton Class of 2025 statistics and Plot the confusion Matrix using seaborn library, MBA dataset.

ALGORITHM:

1. Import required libraries such as pandas, seaborn, matplotlib, sklearn, XGBoost, and LightGBM for data processing, modeling, and visualization.
2. Load the dataset MBA.csv into a dataframe and clean column names.
3. Identify the target column (Admission) dynamically from the dataset.
4. Filter the dataset to keep only relevant classes (Admit and Waitlist).
5. Convert the target variable into binary format (0 = Waitlist, 1 = Admit).
6. Encode categorical features into numerical format using Label Encoding.
7. Separate the dataset into features (X) and target (y).
8. Split the dataset into training and testing sets using an 80:20 ratio.
9. Train classification models using XGBoost and LightGBM algorithms.
10. Evaluate the models using confusion matrix, classification report, accuracy score, and visualize results using heatmaps.

PROGRAM:

Import Libraries

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
```

Load Dataset and Clean Columns


```

df = pd.read_csv("MBA.csv")
df.columns = df.columns.str.strip()

# Find Admission Column

target_col = None

for col in df.columns:
    if col.lower() == 'admission':
        target_col = col
        break

if target_col is None:
    raise ValueError("Admission column not found")

# Filter Required Classes

df[target_col] = df[target_col].astype(str).str.strip().str.lower()
df = df[df[target_col].isin(['admit', 'waitlist'])]

# Convert Target to Binary

df['Status'] = df[target_col].map({
    'waitlist': 0,
    'admit': 1})

df = df.drop(target_col, axis=1)

# Encode Categorical Data

for col in df.select_dtypes(include='object').columns:
    df[col] = LabelEncoder().fit_transform(df[col])

# Features and Target

X = df.drop('Status', axis=1)
y = df['Status']

# Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train Models (XGBoost & LightGBM)

xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')

```

```
xgb_model.fit(X_train, y_train)
lgb_model = LGBMClassifier()
lgb_model.fit(X_train, y_train)

# Prediction, Evaluation and Visualization

# XGBoost

y_pred_xgb = xgb_model.predict(X_test)
cm_xgb = confusion_matrix(y_test, y_pred_xgb)
plt.figure(figsize=(5,4))
sns.heatmap(cm_xgb, annot=True, fmt='d', cmap='coolwarm',
            xticklabels=['Waitlist','Admit'],
            yticklabels=['Waitlist','Admit'])
plt.title("XGBoost Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

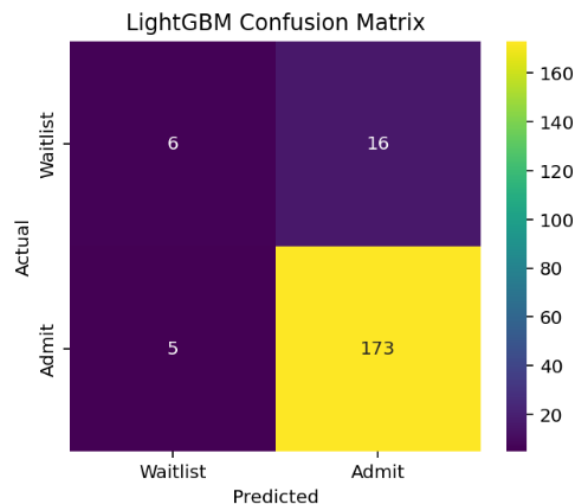
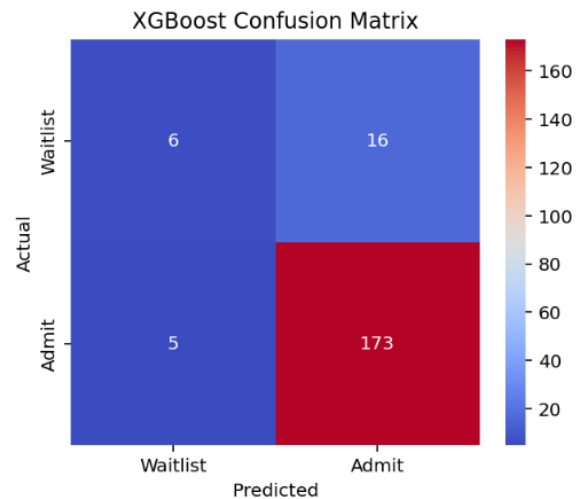
print("\nXGBoost Report:\n")
print(classification_report(y_test, y_pred_xgb))
print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))

# LightGBM

y_pred_lgb = lgb_model.predict(X_test)
cm_lgb = confusion_matrix(y_test, y_pred_lgb)
plt.figure(figsize=(5,4))
sns.heatmap(cm_lgb, annot=True, fmt='d', cmap='viridis',
            xticklabels=['Waitlist','Admit'],
            yticklabels=['Waitlist','Admit'])
plt.title("LightGBM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
print("\nLightGBM Report:\n")
print(classification_report(y_test, y_pred_lgb))
print("LightGBM Accuracy:", accuracy_score(y_test, y_pred_lgb))
```

OUTPUT:



RESULT: Thus the program to implement XGBoost and LightGBM technique to Classify Waitlist and admit in Wharton Class of 2025 statistics and Plot the confusion Matrix using seaborn library has been successfully executed and the output is verified.

EX. NO.:10

K- MEANS AND HIERARCHIAL CLUSTERING ALGORITHMS

DATE:

AIM: To implement K-means and Hierarchical Clustering on customer segmentation dataset and plot the elbow and dendrogram for k-means and Hierarchical clustering respectively using credit_risk_dataset.

ALGORITHM:

1. Import required libraries such as pandas, numpy, matplotlib, sklearn, and scipy for data processing, clustering, and visualization.
2. Load the dataset credit_risk_dataset.csv into a dataframe.
3. Preprocess the data by removing missing values.
4. Convert categorical columns into numerical format using Label Encoding.
5. Separate the dataset into features (X) and target variable (loan_status).
6. Scale the features using StandardScaler for better clustering performance.
7. Apply the Elbow Method to determine the optimal number of clusters for K-Means.
8. Perform K-Means clustering and map clusters to actual labels for evaluation.
9. Apply Hierarchical clustering using linkage and visualize clusters using a dendrogram.
10. Evaluate both clustering methods using accuracy score and classification report.

PROGRAM:

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import classification_report, accuracy_score
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
```

Load Dataset

```
df = pd.read_csv("/mnt/data/credit_risk_dataset.csv")
```

Data Preprocessing

```
df = df.dropna()
```

Encode Categorical Columns

```
le = LabelEncoder()
```

```
for col in df.select_dtypes(include=['object']).columns:
```

```
    df[col] = le.fit_transform(df[col])
```

Target Variable Separation

```
target = 'loan_status'
```

```
X = df.drop(columns=[target])
```

```
y = df[target]
```

Feature Scaling

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Elbow Method for K-Means

```
wcss = []
```

```
for i in range(1, 11):
```

```
    kmeans = KMeans(n_clusters=i, random_state=42)
```

```
    kmeans.fit(X_scaled)
```

```
    wcss.append(kmeans.inertia_)
```

```
plt.figure()
```

```
plt.plot(range(1, 11), wcss, marker='o')
```

```
plt.title("Elbow Method (K-Means)")
```

```
plt.xlabel("Number of Clusters")
```

```
plt.ylabel("WCSS")
```

```
plt.show()
```

Apply K-Means and Evaluate

```
kmeans = KMeans(n_clusters=3, random_state=42)
```

```
clusters_kmeans = kmeans.fit_predict(X_scaled)
```

```

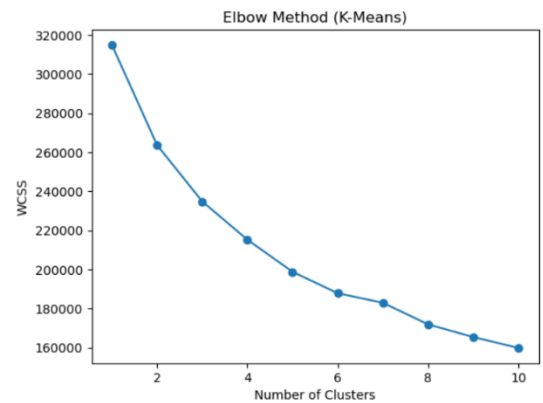
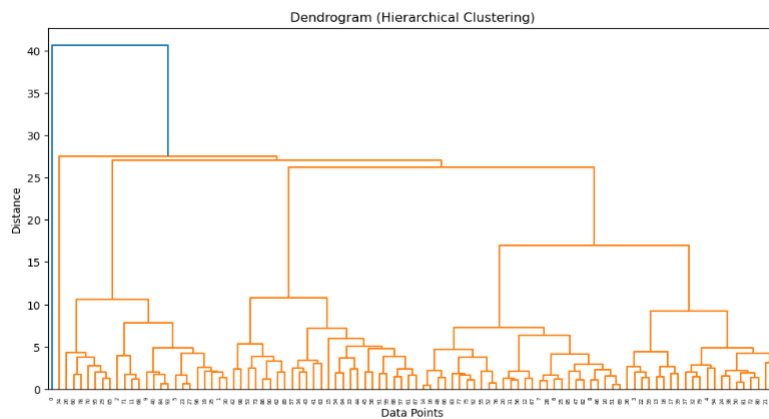
mapping = {}
for i in np.unique(clusters_kmeans):
    mapping[i] = y[clusters_kmeans == i].mode()[0]
y_pred_kmeans = [mapping[c] for c in clusters_kmeans]
print("\n===== K-MEANS RESULTS =====")
print("Accuracy:", accuracy_score(y, y_pred_kmeans))
print("Classification Report:\n", classification_report(y, y_pred_kmeans))

# Hierarchical Clustering with Dendrogram
linked = linkage(X_scaled[:100], method='ward')
plt.figure(figsize=(12, 6))
dendrogram(linked)
plt.title("Dendrogram (Hierarchical Clustering)")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

# Apply Hierarchical Clustering and Evaluate
clusters_hier = fcluster(linked, t=3, criterion='maxclust')
y_subset = y.iloc[:100]
mapping_hier = {}
for i in np.unique(clusters_hier):
    mapping_hier[i] = y_subset[clusters_hier == i].mode()[0]
y_pred_hier = [mapping_hier[c] for c in clusters_hier]
print("\n===== HIERARCHICAL RESULTS =====")
print("Accuracy:", accuracy_score(y_subset, y_pred_hier))
print("Classification Report:\n", classification_report(y_subset, y_pred_hier))

```

OUTPUT:



```
===== K-MEANS RESULTS =====
Accuracy: 0.7833996787485159
Classification Report:
              precision    recall  f1-score   support

     0       0.78         1.00         0.88     22435
     1       0.00         0.00         0.00       6203

   accuracy          0.78         0.78         0.78     28638
  macro avg          0.39         0.50         0.44     28638
 weighted avg          0.61         0.78         0.69     28638
```

```
===== HIERARCHICAL RESULTS =====
Accuracy: 0.59
Classification Report:
              precision    recall  f1-score   support

     0       1.00         0.02         0.05        42
     1       0.59         1.00         0.74         58

   accuracy          0.59         0.59         0.59       100
  macro avg          0.79         0.51         0.39       100
 weighted avg          0.76         0.59         0.45       100
```

RESULT: Thus the program to implement K-means and Hierarchical Clustering on customer segmentation dataset and plot the elbow and dendrogram for k-means and Hierarchical clustering respectively has been successfully executed and the output is verified.